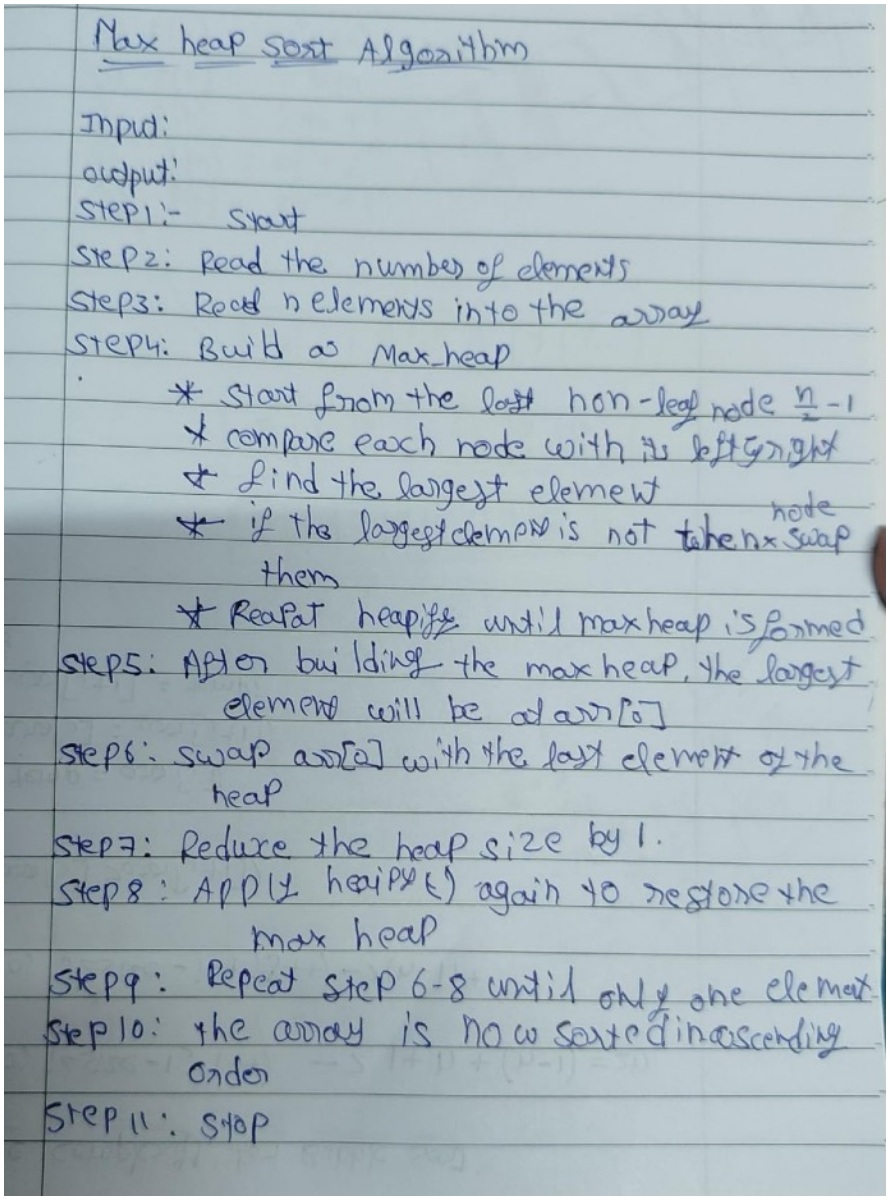
PRACTICAL-3

AIM: To implement the Max-Heap Sort algorithm for sorting the given elements in ascending order.

Max-Heap Sort algorithm:



Max heap sort Algorithm

Input:
output:
STEP1:- Start
Step2: Read the number of elements
Step3: Read n elements into the array
STEP4: Build a Max_heap
* Start from the last non-leaf node $\frac{n}{2}-1$
* compare each node with its left & right
* find the largest element
* if the largest element is not the node swap them
* Repeat heapify until max heap is formed
Step5: After building the max heap, the largest element will be at arr[0]
Step6: swap arr[0] with the last element of the heap
Step7: Reduce the heap size by 1.
Step8: Apply heapify() again to restore the max heap
Step9: Repeat step 6-8 until only one element
Step10: the array is now sorted in ascending order
Step11: Stop

PROGRAM/CODE:

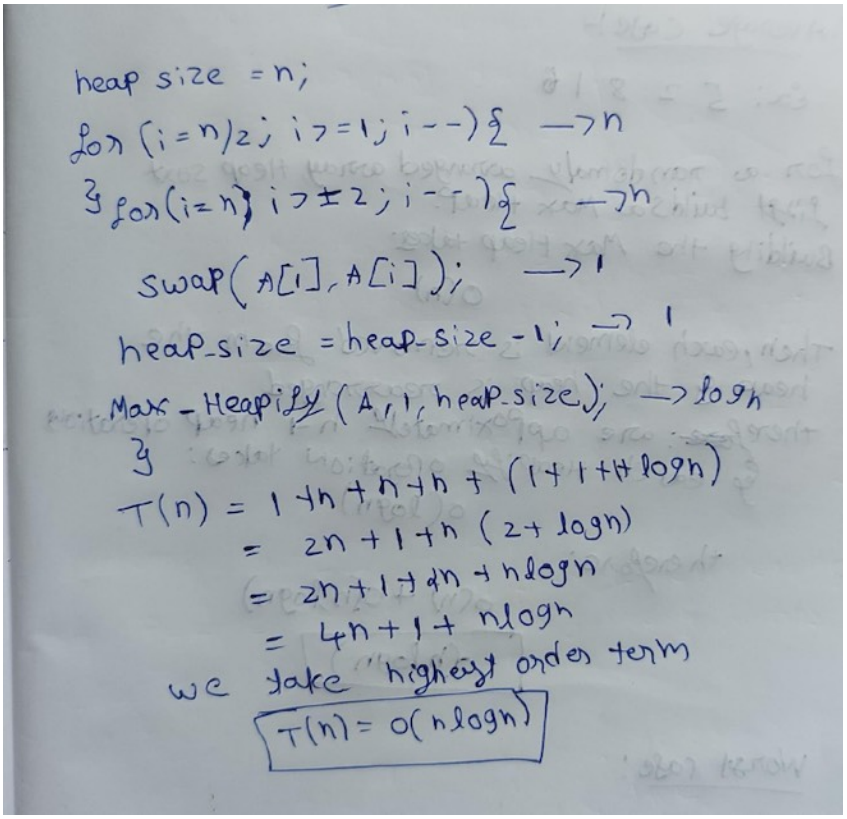
```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && arr[left] > arr[largest])
      largest = left;

   if (right < n && arr[right] > arr[largest])
      largest = right;

   if (largest != i)
   {
      swap(arr[i], arr[largest]);
      heapify(arr, n, largest);
   }
}

void heapSort(int arr[], int n)
{
   // Build Max Heap
   for (int i = n / 2 - 1; i >= 0; i--)
      heapify(arr, n, i);

   // Heap Sort
   for (int i = n - 1; i > 0; i--)
   {
      swap(arr[0], arr[i]);
      heapify(arr, i, 0);
   }
}

int main()
{
   int n;
   int arr[100];

   cout << "Enter size of array: ";
   cin >> n;
```

```
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

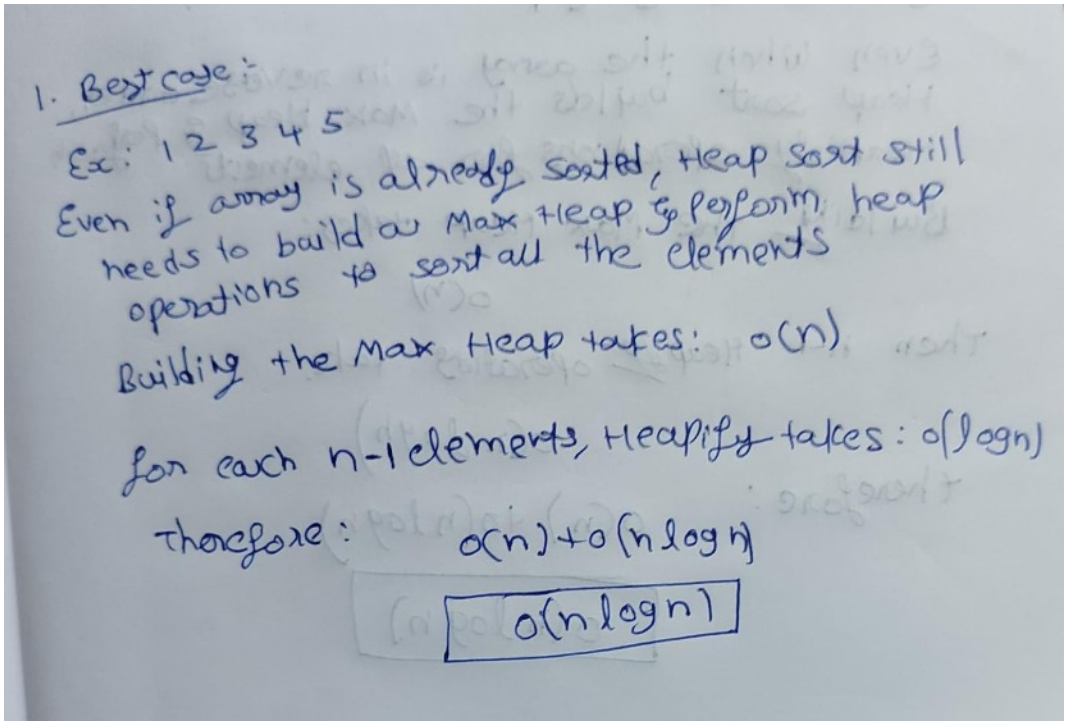
OUTPUT:

```
nandhagopal@NGKs-MacBook-Air c++ % "/Users/nandhagopal/Documents/CODING/c++/MAX-HEAP SORT"
Enter size of array: 4
Enter array elements: 22
11
44
33
Sorted array: 11 22 33 44 %
nandhagopal@NGKs-MacBook-Air c++ %
```

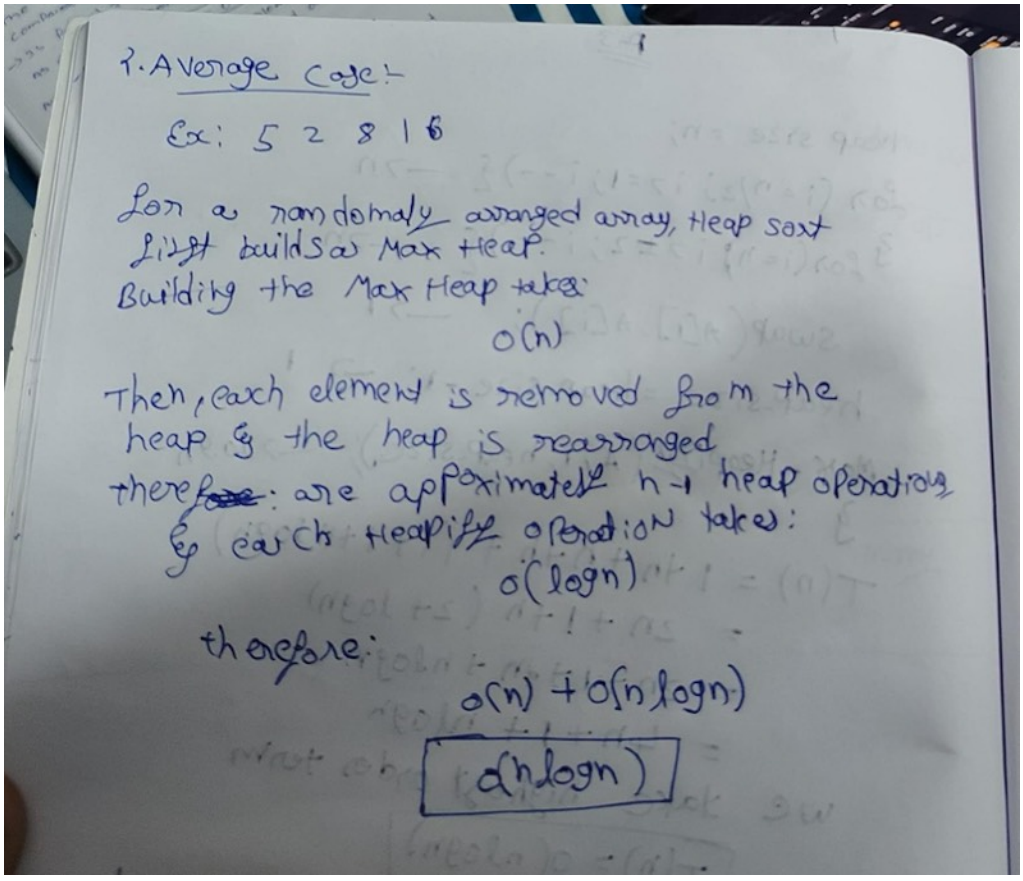
TIME COMPLEXITY:



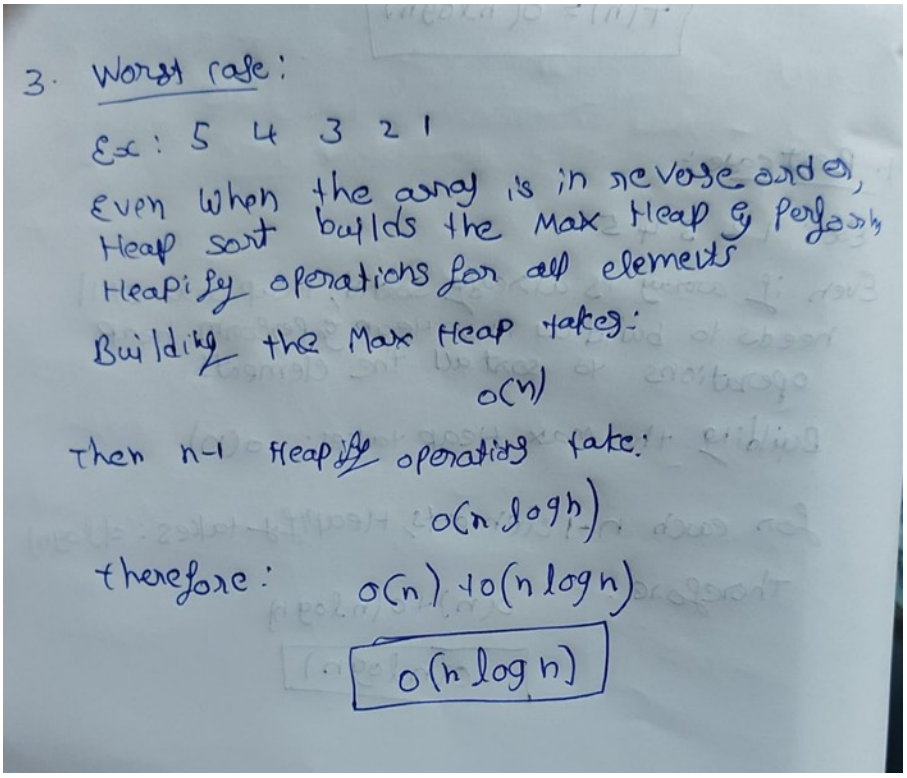
1.BEST CASE:



2.AVERAGE CASE:



3.WROST CASE:



Best Case: (O(nlog n))
Average Case: (O(nlog n))
Worst Case: (O(nlog n))

CONCLUSION:

Thus, the Max-Heap Sort algorithm was successfully implemented, and the given array elements were sorted in ascending order using the Max Heap concept.